Sentiment Intelligence

Dashboard

Complete Project Documentation

Version 4.0 Pro

Tech Stack: Python | Streamlit | FastAPI | DistilBERT | Redis | Docker

Author: Sanjay Kumar

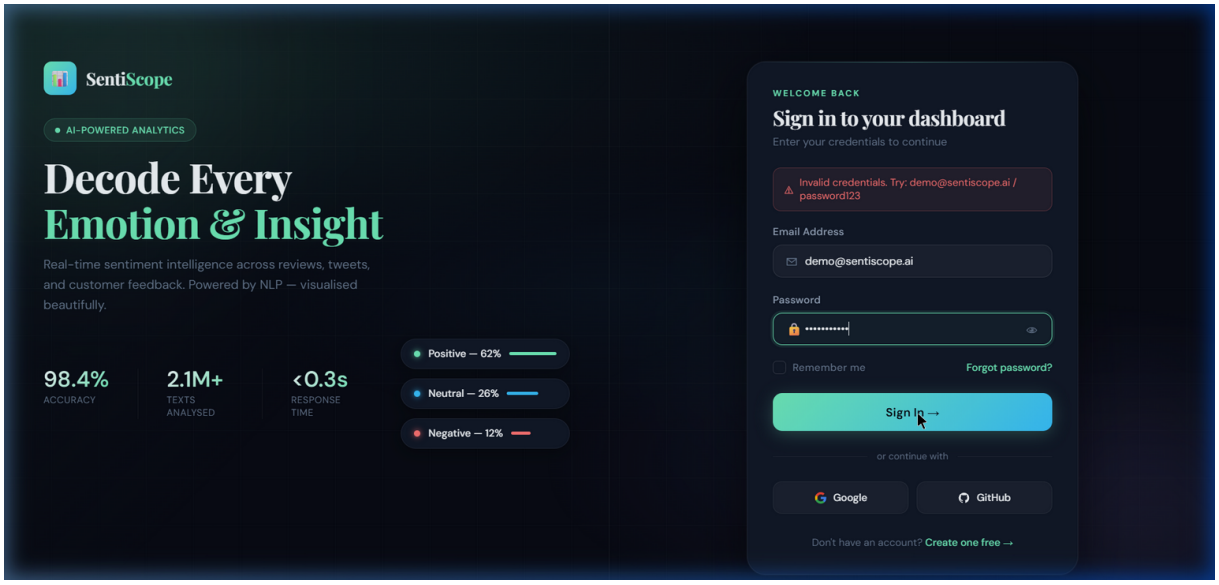


Table of Contents

1. Project Overview

2. Architecture & Tech Stack

3. Project Structure

4. Installation & Setup

5. Dashboard Pages

  5.1 Login & Authentication

  5.2 Home Page

  5.3 Overview & KPIs

  5.4 Platform Analysis

  5.5 Live Predictor

  5.6 Competitive Benchmarking

  5.7 Model Insights

  5.8 Export & Reporting

6. Data Pipeline

7. ML Model Details

8. API Service

9. Monitoring & Drift Detection

10. Theming System

11. Configuration Reference

1. Project Overview

Sentiment Intelligence is a production-grade, end-to-end sentiment analysis platform that combines a fine-tuned DistilBERT NLP model with an interactive Streamlit dashboard. It analyzes social media engagement data across 6 major platforms (Instagram, TikTok, Twitter, YouTube, LinkedIn, Facebook) and provides deep insights through interactive charts, real-time ML predictions, SHAP-based explainability, and automated PDF reporting.

Key Capabilities

- 6 interactive analytics pages with Plotly visualizations
- Rea

2. Architecture & Tech Stack

The system follows a layered architecture: Data Ingestion -> Feature Engineering -> ML Training -> FastAPI Inference -> Streamlit Dashboard. Each layer is independently deployable via Docker containers.

Tech Stack

Layer	Technology
Frontend	Streamlit 1.40+, Plotly, Custom HTML/CSS/JS
Backend API	FastAPI, Uvicorn, Pydantic
ML Model	HuggingFace Transformers (DistilBERT), PyTorch
Explainability	SHAP
Tracking	MLflow
Caching	Redis 7
Data	Pandas, PyArrow (Parquet)
PDF	FPDF2, Matplotlib
Monitoring	Evidently AI, Prometheus
Containers	Docker, Docker Compose

3. Project Structure

```
Sentiment Analysis Dashboard/
|-- config.yaml              (central config)
|-- docker-compose.yml       (multi-service orchestration)
|-- generate_data.py         (synthetic dataset generator)
|-- requirements.txt         (Python dependencies)
|-- .env.example             (environment template)
|-- data/
|   |-- raw/                 (CSV datasets)
|   |-- processed/           (cleaned Parquet)
|   |-- artifacts/           (quality reports)
|-- saved_model/
|   |-- model_card.json      (model metadata)
|-- src/
|   |-- api/
|   |   |-- main.py          (FastAPI endpoints)
|   |   |-- client.py        (HTTP client)
|   |   |-- middleware.py    (rate limit, CORS)
|   |   |-- schemas.py      (Pydantic models)
|   |-- data/
|   |   |-- loader.py        (CSV->Parquet)
|   |   |-- cleaner.py       (dedup, missing)
|   |   |-- feature_engineer.py
|   |-- models/
|   |   |-- trainer.py       (DistilBERT fine-tuning)
|   |-- monitoring/
|   |   |-- drift_detector.py
|   |   |-- retrain_trigger.py
|   |-- dashboard/
|       |-- app.py           (main entry)
|       |-- components/      (10 UI components)
|       |-- pages/           (6 analytics pages)
```

4. Installation & Setup

Prerequisites

- Python 3.10+
- pip (

Quick Start

```
1. git clone <repo-url>
2. cd "Sentiment Analysis Dashboard"
3. python -m venv venv
4. venv\Scripts\activate        (Windows)
5. pip install -r requirements.txt
6. cp .env.example .env
7. python generate_data.py
8. python -m src.data.loader
9. streamlit run src/dashboard/app.py
```

The dashboard will be available at http://localhost:8501

Docker Deployment

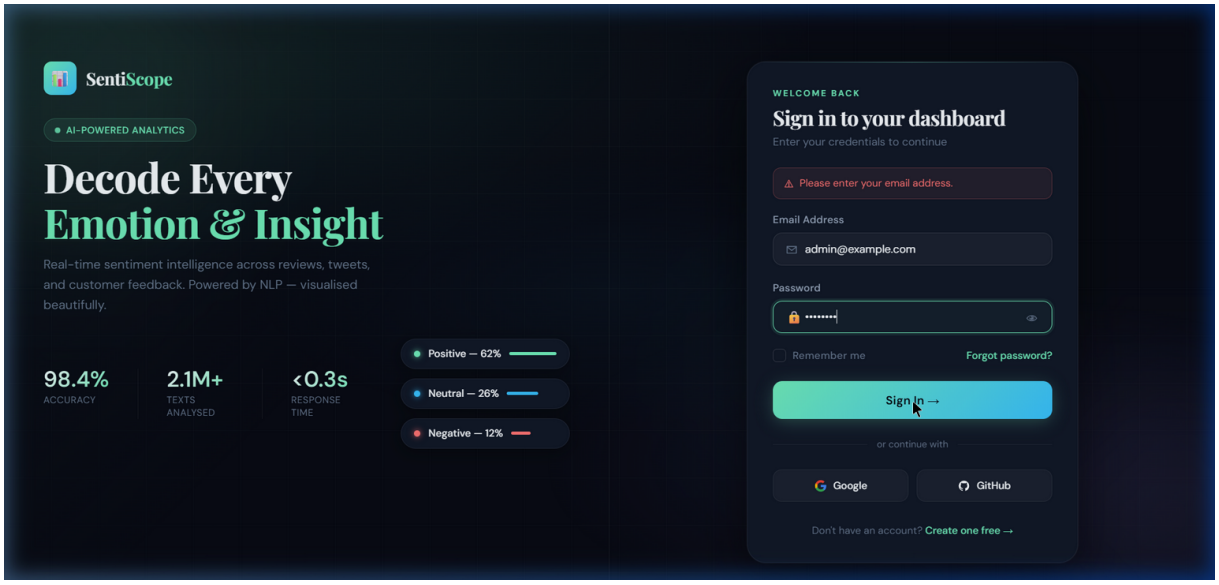
```
docker-compose up --build

Dashboard: http://localhost:8501
API:       http://localhost:8000
MLflow:    http://localhost:5000
Redis:     localhost:6379
```

5. Dashboard Pages

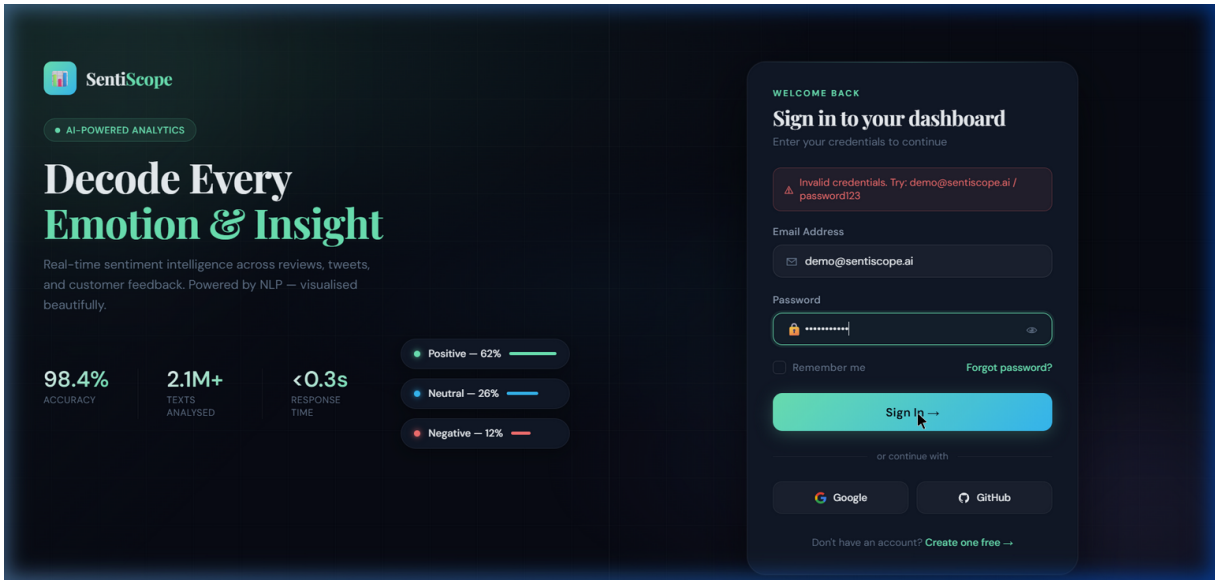
5.1 Login & Authentication

Custom SentiScope branded login with email/password. Session persistence via st.session_state.



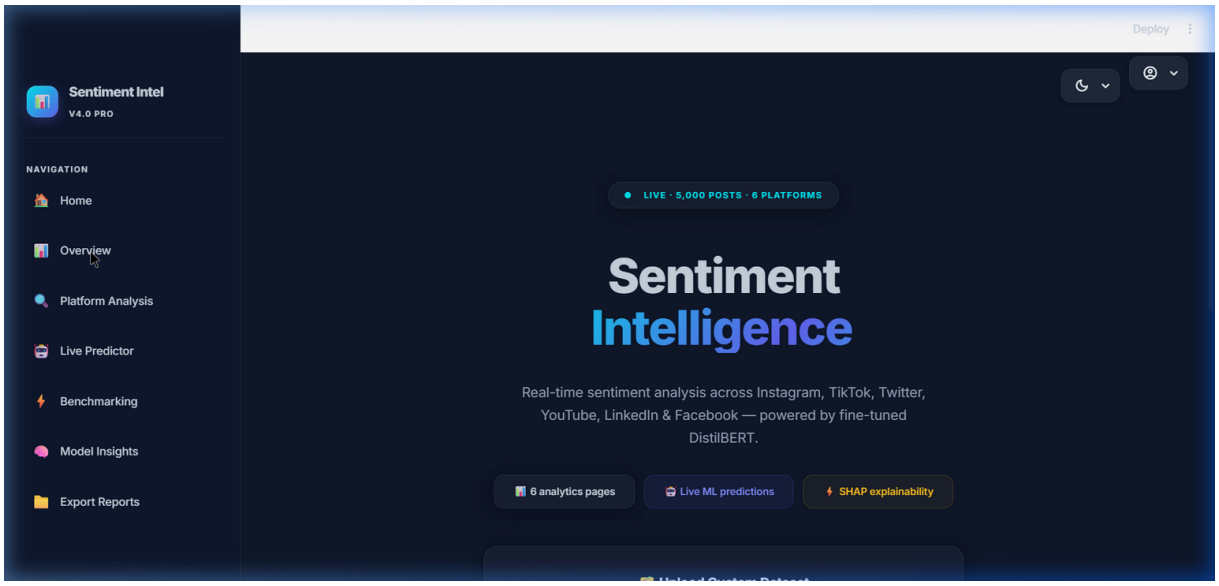
5.2 Home Page

Landing page with animated hero section, live statistics badge, feature highlights, dataset upload with modal progress popup, and sentiment metrics grid.



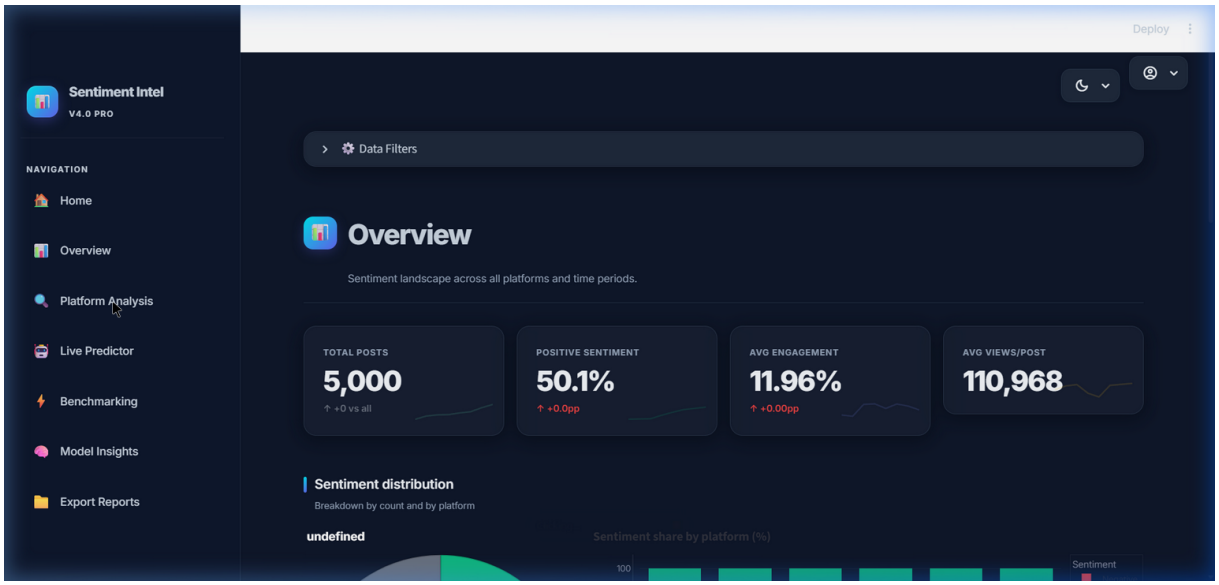
5.3 Overview & KPIs

- 4 animated KPI cards with delta indicators
- Sen



5.4 Platform Analysis

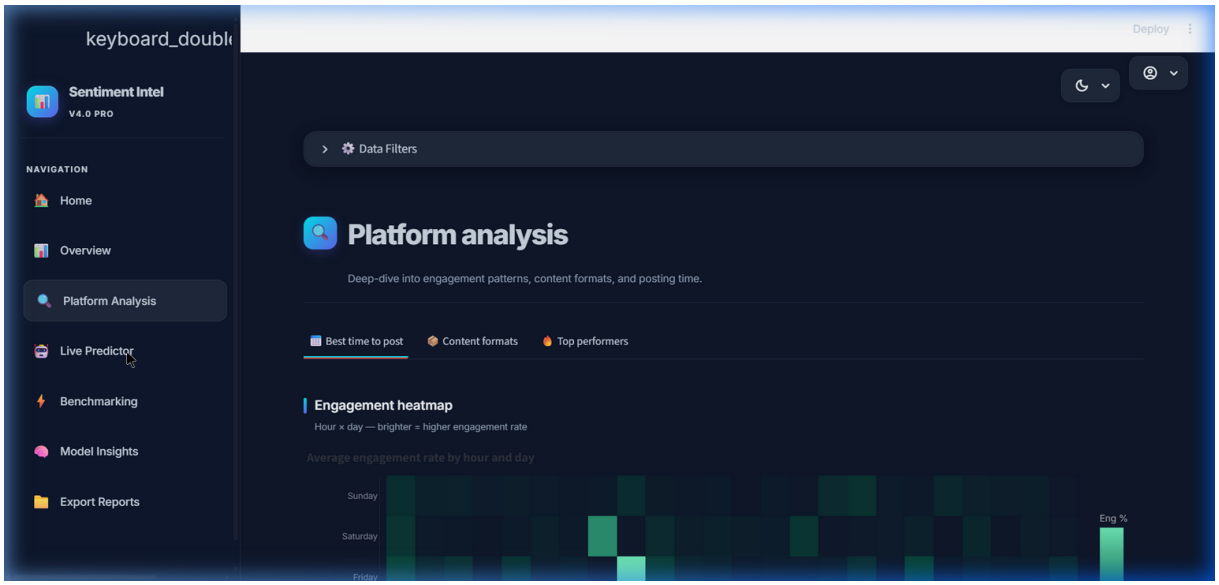
- Engagement heatmap (hour x day) for optimal posting
- Rea



5.5 Live Predictor

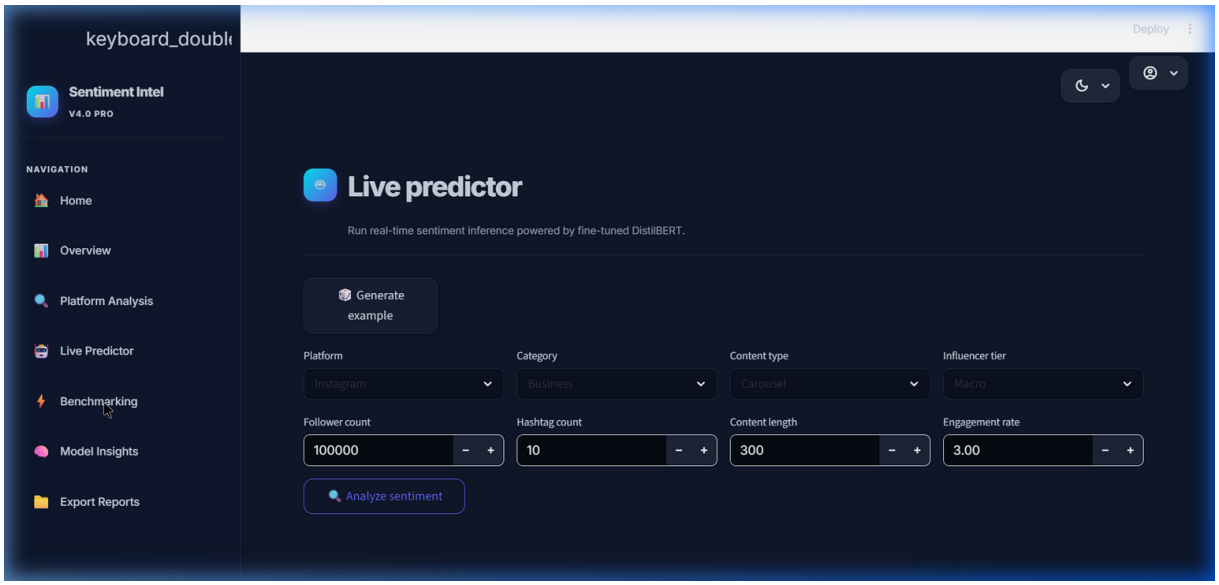
- 8-field input form for real-time inference

- Gen



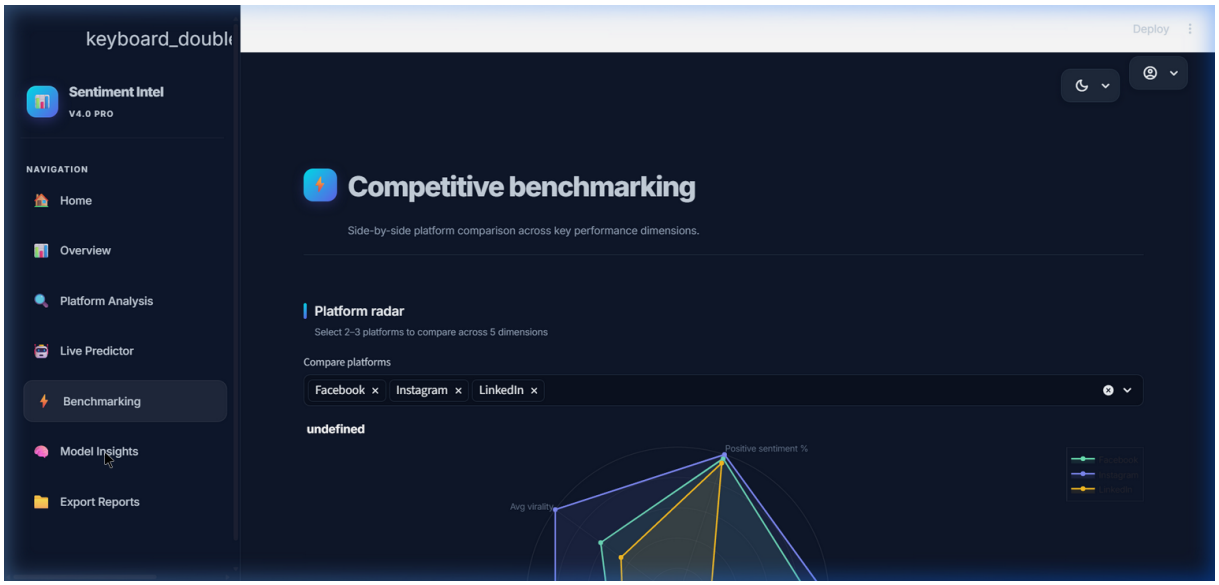
5.6 Competitive Benchmarking

- 5-axis radar chart comparing 2-4 platforms
- Sen



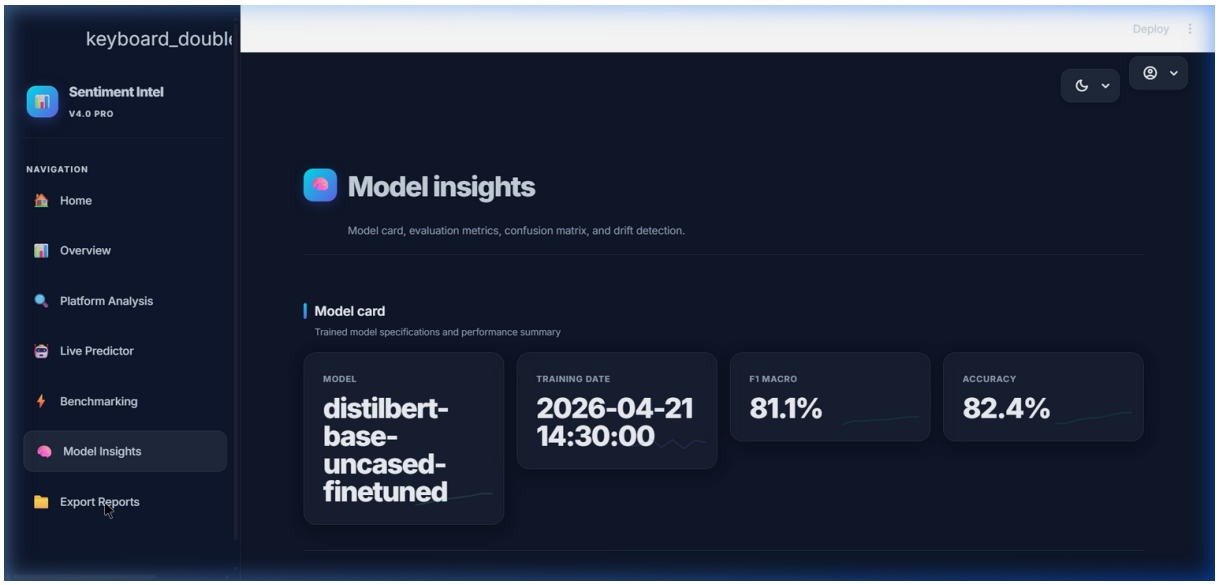
5.7 Model Insights

- Model card (name, date, F1, accuracy)
- Clas



5.8 Export & Reporting

- CSV download of filtered dataset (18 columns + label)
- Auto



6. Data Pipeline

Three-stage pipeline: Ingestion -> Cleaning -> Feature Engineering

Stage	Module	Description
1	loader.py	CSV read, schema validation, Parquet conversion
2	cleaner.py	Dedup by Post_ID, handle missing values, flag outliers
3	feature_engineer.py	Virality score, save rate, time features, buckets

Dataset: 20 columns, 5000+ rows covering 6 platforms, 7 categories, 5 content types, 5 influencer tiers, 3 sentiment classes.

7. ML Model Details

Fine-tuned DistilBERT (distilbert-base-uncased) for 3-class sentiment classification.

Training Config

Parameter	Value
Epochs	5
Batch (train)	16
Batch (eval)	32
Learning rate	2e-5
LR scheduler	Cosine
Warmup	0.1
Weight decay	0.01
Best metric	F1 macro

Performance

Class	Prec	Recall	F1	Support
Positive	0.85	0.89	0.87	3150
Neutral	0.73	0.70	0.71	1150
Negative	0.81	0.77	0.79	750
Accuracy			82.4%	5050

8. API Service

FastAPI backend at port 8000 with /predict and /health endpoints.

Features

- Redis caching (TTL 1hr)
- Rate

9. Monitoring & Drift Detection

PSI-based data drift detection with configurable thresholds. Auto-retraining when F1 < 0.75.

Module	Purpose
drift_detector.py	PSI computation between reference and production data
retrain_trigger.py	Auto-retrain when F1 macro drops below threshold

10. Theming System

600+ lines of CSS in theme_provider.py. Dual Dark/Light mode with seamless switching.

Token	Dark	Light
--bg-main	#0F172A	#F9FAFB
--bg-secondary	#1E293B	#FFFFFF
--text-primary	#F1F5F9	#111827
--brand-primary	#00E6F0	#00E6F0
--brand-secondary	#6366F1	#6366F1

Font: Inter (Google Fonts). Components styled: sidebar, inputs, tabs, expanders, buttons, tables, charts, scrollbar, popover menus, progress bars, metrics cards.

11. Configuration Reference

config.yaml

```yaml
paths:
  raw_data: "data/raw/social_media_engagement_dataset.csv"
  processed_data: "data/processed/processed_data.parquet"
  model_card: "saved_model/model_card.json"

model:
  base_model: "distilbert-base-uncased"
  num_labels: 3
  max_length: 128

training:
  num_train_epochs: 5
  learning_rate: 2.0e-5
  metric_for_best_model: "f1_macro"

app:
  redis_ttl: 3600
  rate_limit: "100/minute"
  drift_threshold_psi: 0.2
  retrain_f1_threshold: 0.75
```

Environment Variables

Variable	Description
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection (default: redis://localhost:6379)
JWT_SECRET_KEY	Secret for JWT signing
MLFLOW_TRACKING_URI	MLflow server URL
API_BASE_URL	FastAPI URL (for Docker networking)